Introducción Parte 2

Funciones -Valor y Referencia Argumentos .
, p/linea de comando Cadenas en C#

Funciones, Módulos o Procedimientos

Las funciones también se denominan módulos o procedimientos.

En lugar de escribir un único programa principal, buscamos podemos dividir la función principal para:

- partes pequeñas de tamaño manejable
- separar las tareas repetitivas
- separar tareas puntuales.

Lo normal para un módulo es que contenga un grupo de funciones para un objetivo específico

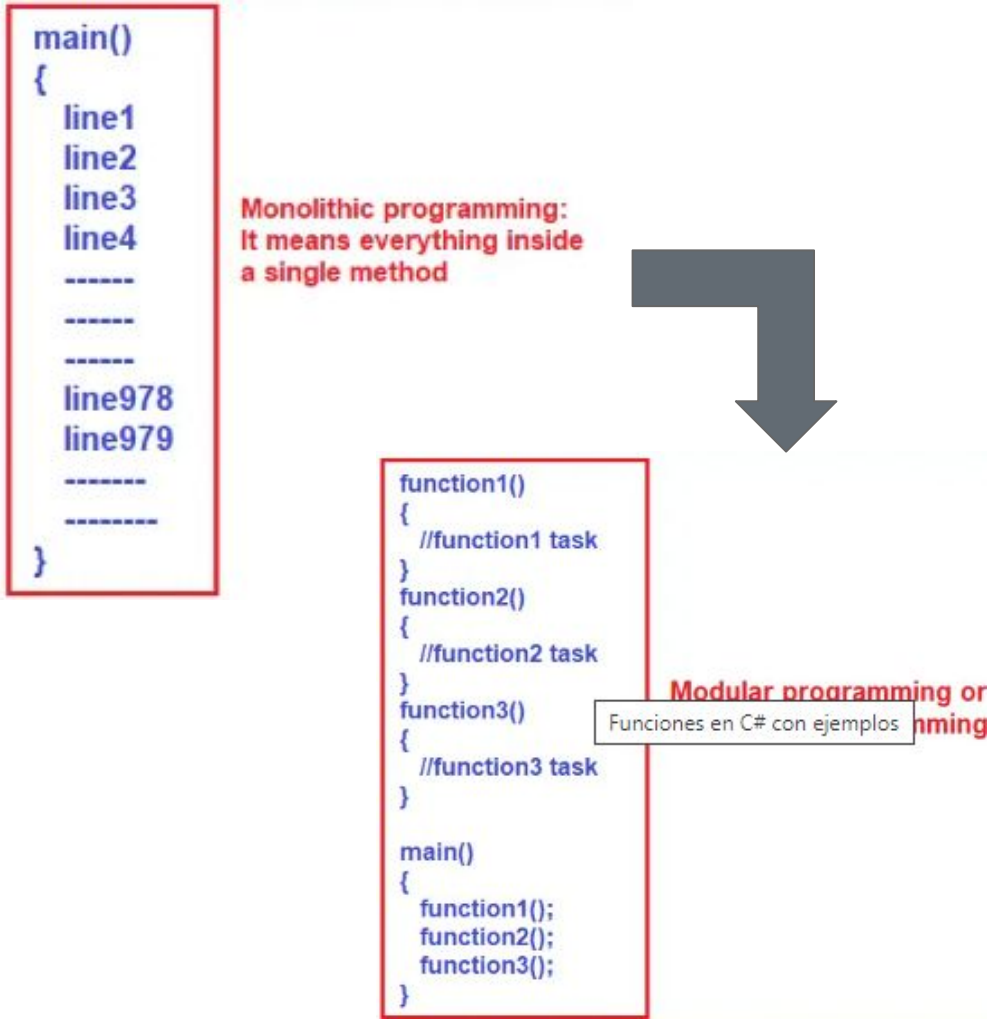
Problemas en la programación monolítica:

- un error en una sola línea, es un error en todo el programa
- 10000 líneas de código no podemos terminarlas en una hora o en un día
- ¿Cuántas personas pueden escribir esta única función principal?
- Cuanta Memoria Necesita

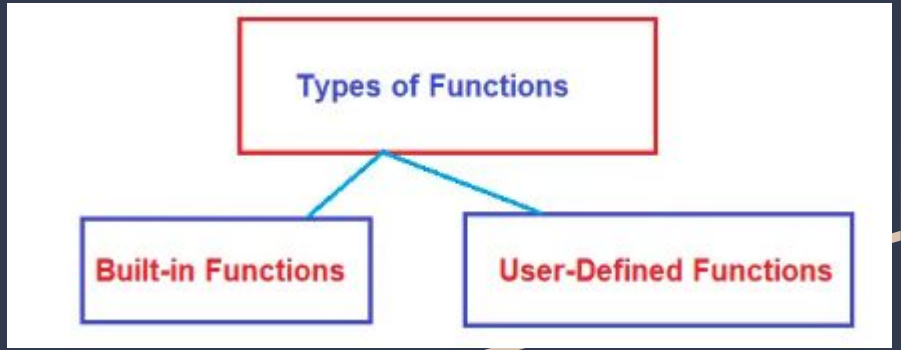
Enfoque

“El enfoque de programación modular, puede dividir el programa en tareas más pequeñas, concentrarse en cuestiones puntuales, terminarlas y hacerlas perfectas.”

Es fácil para una sola persona desarrollar la aplicación, incluso si puede dividir este proyecto de software en un equipo de programadores donde cada programador se concentrará en una o muchas tareas más pequeñas.



Tipos de funciones en C#:



Ventajas de las funciones de biblioteca estándar en C#:

1. Simplemente porque funcionan.
2. Están optimizadas para el rendimiento.
3. Ahorra tiempo de desarrollo.

Ventajas de las funciones definidas por el usuario en C#:

El código de la aplicación será más fácil de entender, mantener y depurar.

Podemos escribir el código una vez y podemos utilizarlo en muchos lugares, *reutilización del código*.

Tamaño del programa reducido.

Crear una función

```
<accessSpecifier> [Modifier] <ReturnType>
<FunctionName> ([ParemeterList])

{

  // FunctionBody

}
```

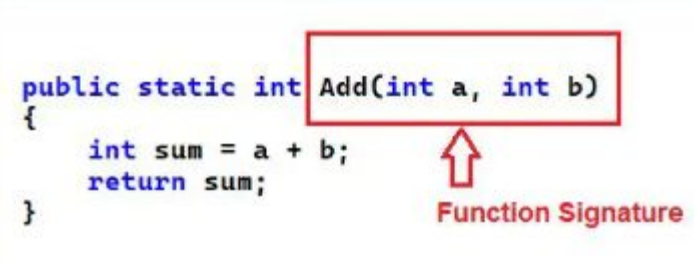
1. **Nombre de la función:** Es obligatorio y define el nombre del método o función. La firma del método consta del nombre del método y la lista de parámetros.
2. **Lista de parámetros:** es opcional.
3. **Tipo de retorno:** es obligatorio y define el valor del tipo de retorno del método.
4. **Especificador de acceso:** es opcional y define el alcance del método.
5. **Modificador:** Es opcional y define el tipo de acceso del método.
6. **Cuerpo de la función:** el cuerpo de la función define el código o la lista de declaraciones que necesita para ejecutar la llamada a la función.

Que es la Signature(firma)

La firma de una función o método consta de dos cosas:

- el nombre
- la lista de parámetros

El tipo de retorno no se considera parte de la firma del método.



Ejemplo de uso:

```
using System;
namespaces FuncionDemo
{
  class Program
  {
    static void Main( cadena [] args )
    {
      entero x, y;
      x = 10 ;
      y = 15 ;
      int suma = Sumar( x, y ) ;
      Console.WriteLine($"La suma es
{suma}");
      Console.ReadKey() ;
    }


    int static Sumar(int a,int b)
    {
        suma int = a + b;
        suma de devolución ;
    }
  }
}
```

Llamada a la Función

Signature de la Función

Funciones Definidas por el usuario

Hay cuatro tipos de funciones definidas por el usuario en C#. Son los siguientes:

- Funciones sin argumento y sin tipo de retorno.
- Funciones con argumento y sin tipo de retorno.
- Funciones sin argumento y con tipo de retorno.
- Funciones con argumento y con tipo de retorno

Veamos los ejemplos

Funciones sin argumento y sin tipo de retorno

```
using System;
namespace FunctionDemo
{
    class Program
    {
        static void Main(string[]
args)
        {
            Sum();
            Console.ReadKey();
        }

        static void Sum()
        {
            int x = 10;
            int y = 20;
            int sum = x + y;
            Console.WriteLine($"Suma
de {x} y {y} es {sum}");
        }
    }
```

Funciones sin argumento y con tipo de retorno

```
using System;
namespace FunctionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Result=Sum();
            Console.WriteLine($"Suma
es {Result}");
            Console.ReadKey();
        }


        static int Sum()
        {
            int x = 10;
            int y = 20;
            int sum = x + y;
            return sum;
        }
    }
}
```

Funciones con argumento y sin tipo de retorno

```
using System;
namespace FunctionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 10, y = 20;
            Sum(x, y);
            Console.ReadKey();
        }

        static void Sum(int x, int y)
        {
            int sum = x + y;
            Console.WriteLine($"Suma es
{sum}");
        }
    }
}
```

Funciones con argumento y con tipo de retorno

```
using System;
namespace FunctionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 10, y = 20;
            int Result = Sum(x, y);
            Console.WriteLine($"Suma es
{Result}");
            Console.ReadKey();
        }

        static int Sum(int x, int y)
        {
            int sum = x + y;
            return sum;
        }
    }
}
```

Sobrecarga de funciones en C#

Podemos escribir más de una función con el mismo nombre, si y solo si:

con **nro** de argumento **distintos**

o

con **tipo** de argumentos **distintos**

Ventajas

- Unifica nombres para escribir funciones que realizan la misma actividad.
- Facilita la recordar los nombres

Veamos un Ejemplo

Ejemplo Sobrecarga de funciones

```
using System;
namespace FunctionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 10, b = 2, c, d;
            c = add(a, b);
            Console.WriteLine($"Suma de {a} and {b} is {c}" );
            d = add(a, b, c);
            Console.WriteLine($"Suma de {a} y {b} y {c} is {d}" );
            Console.WriteLine($"Suma de 10.5 y 25.6 es {add(10.5f,
25.6f)}");
            Console.ReadKey();
        }
        static int add(int x, int y)
        {
            return x + y;
        }
        static int add(int x, int y, int z)
        {
            return x + y + z;
        }
        static float add(float x, float y)
        {
            return x + y;
        }
    }
}
```

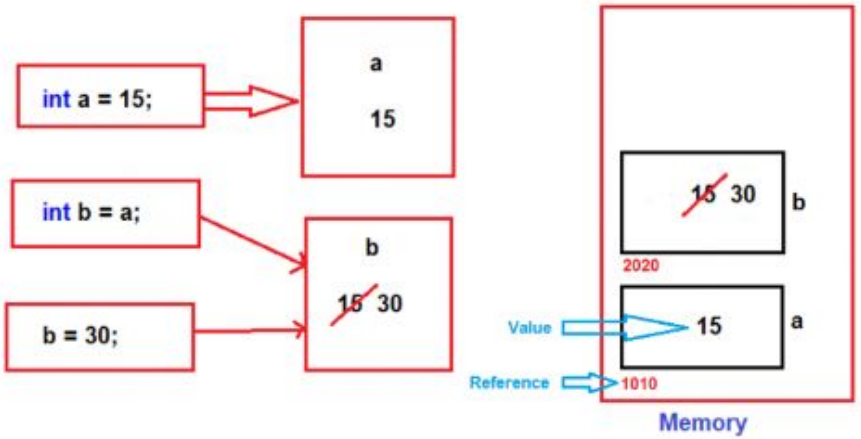
Llamar por valor y llamar por referencia

"¿Qué es Call/Pass por valor y Qué es Call/Pass por referencia?"

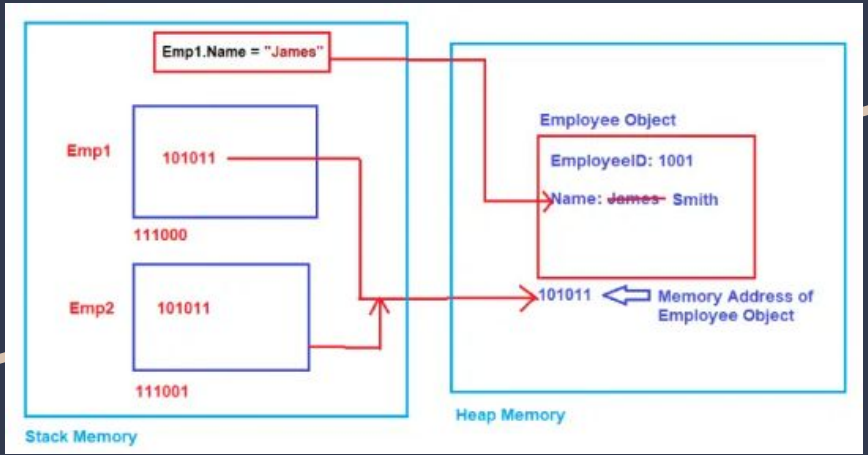
Llamar por valor en C#

En .NET Framework, de forma predeterminada, todos los objetos se llaman por valor, no por referencia . Ya sea un tipo de valor o un tipo de datos de referencia

se llamarán por valor de forma predeterminada.



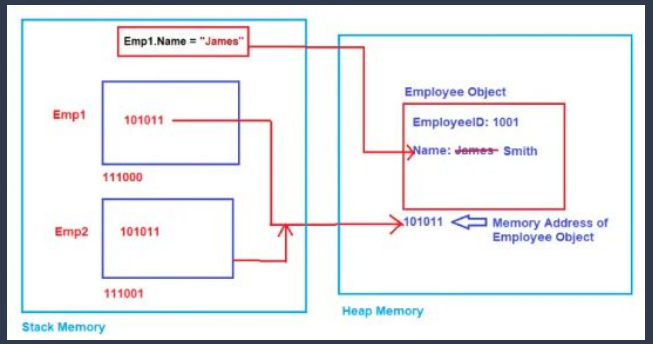
Llamada por valor con tipos de referencia

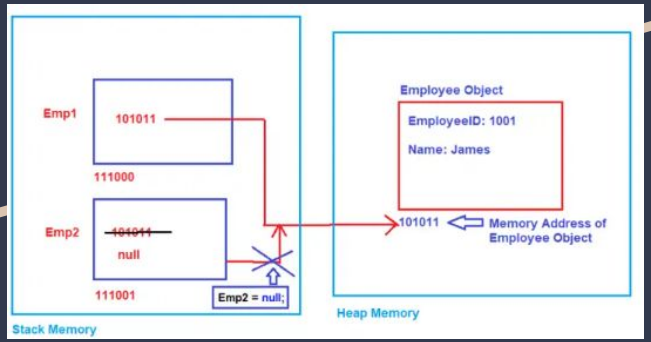



```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Empleado Emp1 = new Empleado();
            Emp1.EmpleadoID = 1001;
            Emp1.Nombre = "James";
            Empleado Emp2 = Emp1;
            Emp1.Name = "Smith;
            Console.WriteLine($"Emp1
Nombre= {Emp1.Name}");
            Console.WriteLine($"Emp2
Nombre= {Emp2.Name}");
            Console.ReadKey();
        }
    }

    public class Empleado
    {
        public int EmpleadoID;
        public string Nombre;
    }
}
```

Qué pasa con los métodos?



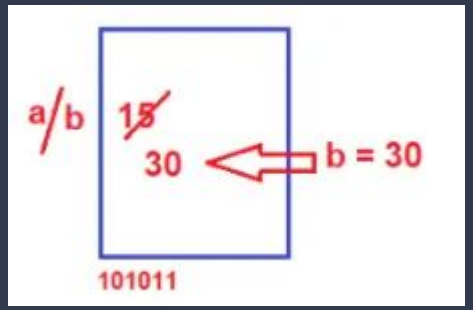



```
static void Main(string[] args)
{
     Employee Emp1 =  new Empleado();
     Emp1.EmpleadoID = 1001;
     Emp1.Nombre = "James";
     UpdateNombre(Emp1);
     Console.WriteLine($"Emp1 Nombre=
{Emp1.Name}");
     Console.ReadKey();
}


public static void UpdateNombre(Empleado Emp2)
{
     Emp2.Nombre = "Smith";
     Console.WriteLine($"Emp2 Nombre=
{Emp2.Name}");
}


public static void UpdateNombre2(Employee Emp2)
{
          Emp2 = null;
}
```

Llamada por referencia en C#

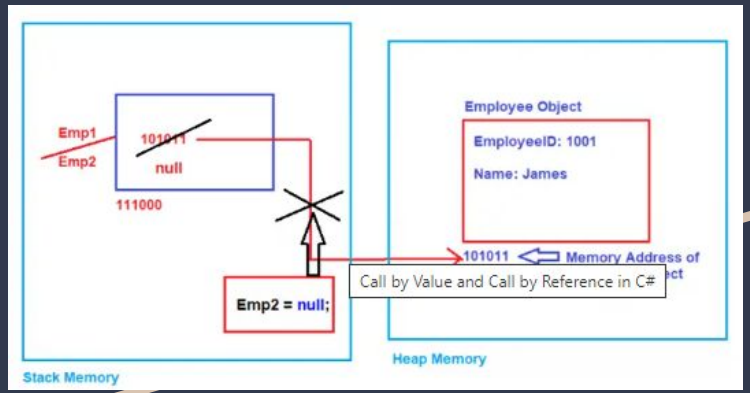


Necesitamos anteponer al parámetro la palabra clave ref para la llamada por referencia

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            UpdateValue(ref a);
            Console.WriteLine(a);
            Console.ReadKey();
        }

        static void UpdateValue(ref int b)
        {
            b = 30;
        }
    }
}
```

Llamada por referencia con tipos de referencia




```
using System;
namespace FunctionsDemo
   {
    class Program
    {
        static void Main(string[] args)
        {
            Employee Emp1 =  new Employee();
            Emp1.EmployeeID = 1001;
            Emp1.Name = "James";
            UpdateName(ref Emp1);
            Console.WriteLine($"Emp1 Name =
{Emp1.Name}");
            Console.ReadKey();
        }
        public static void UpdateName(ref
Employee Emp2)
        {
            Emp2 = null;
        }
    }
    public class Employee
    {
        public int EmployeeID;
        public string Name;
      }
}
```

Diferencia entre llamada por valor y llamada por referencia

Llamada por valor en C#:

1. La copia de la variable original se pasa a la función llamada.
2. El valor del parámetro original se copia en el parámetro de la función. Como resultado, si realizamos modificaciones a los parámetros formales, no tienen ningún impacto en el parámetro real.
3. No le permitirá cambiar las variables reales mediante llamadas a funciones.
4. La ubicación de la memoria a la que se refieren los parámetros formales y los argumentos reales es diferente.
5. No requiere una palabra clave ref o out en C#.

Llamada por referencia en C#:

1. La referencia de la variable (o la dirección real) se pasa a la función llamada.
2. Los parámetros formales y reales apuntan a la misma dirección de memoria. Como resultado, cualquier cambio realizado en los parámetros formales también se refleja en los parámetros reales.
3. Le permite cambiar el valor real de la variable mediante llamadas a funciones.
4. La ubicación de la memoria a la que hacen referencia los parámetros formales y los argumentos reales es la misma.
5. Requiere una palabra clave ref o out para lograr Llamada por referencia en C#.

Argumentos de línea de comando en C#

```
using System;

namespace FirstProgram

{

  class Program

  {

     static void Main(string[] args)

     {

          Console.WriteLine($"Primer argumento de línea
de comando{args[0]}");

         Console.WriteLine($"Segundo argumento de línea
de comando {args[1]}");

         Console.WriteLine($"Tercero argumento de línea
de comando {args[2]}");

         Console.ReadLine();

               }

       }

}
```

arg

- Aplicación (3)
- Compilación (1)
- Paquete (1)
- **Depurar** (2)
  - General (2)

Depurar

General

La administración de perfiles de inicio
puede acceder a él a través del víncul
bien mediante el comando Destino d

Abrir la interfaz de perfiles de inicio d

Perfiles de inicio

FuncionesDemo

Argumentos de la línea de comandos

Argumentos de la línea de comandos que se pasan al ejecutable. Puede dividir a
varias líneas.

```
ale pepe nose
```

Directorio de trabajo

Ruta de acceso al directorio de trabajo en el que se iniciará el proceso.

Examinar...

Usar máquina remota

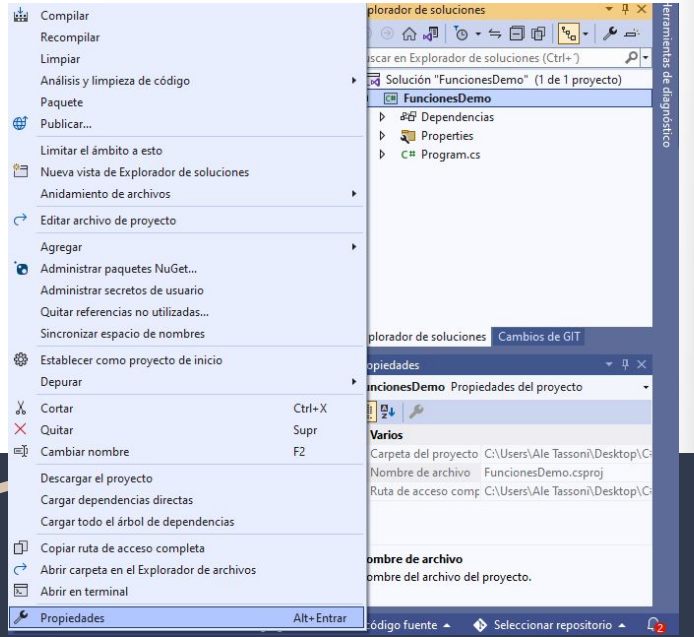
Indica que el depurador debe asociarse a un proceso en una máquina remo

Variables de entorno

Variables de entorno que se establecerán antes de ejecutar el proceso.

Nombre	Valor

Habilitar Recarga activa



Cadenas

Las cadenas son tipos de referencia

Las cadenas en **C# son tipos de referencia**, es decir, no son tipos de datos normales o se puede decir que **no son como otros tipos de datos primitivos**.

Diferencia entre **string** (minuscula) y String (capital).

El **string** es en realidad un alias de String. Si hace clic derecho en la string en minuscula y va a la definición, verá que el nombre de la clase real es una String en capital.

Puede utilizar cualquiera de ellos

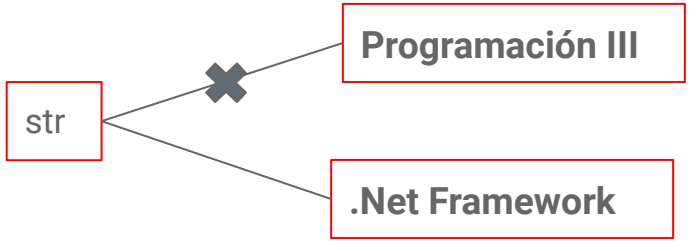
Por nomenclatura, cuando cree una variable, **string** y siempre que desee invocar métodos en la cadena, utilice String.



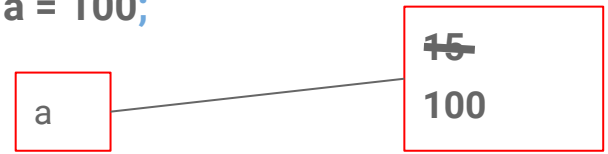
Cadenas

Las cadenas son inmutables

```
string str = " Programación III" ;
str = ".Net Framework"
```




```
int a = 15;
a = 100;
```



Veamos un ejemplo validar la dicho

Cadenas

¿Cómo podemos mejorar el rendimiento utilizando String intern?

Veamos qué pasa si asignamos una cadena fija en el ejemplo

```
str = "UTN - FRRA" + str;
```

si está ejecutando un bucle for y asignando el mismo valor una y otra vez, entonces utiliza la internación de cadenas para mejorar el rendimiento. En este caso, en lugar de crear un objeto nuevo, utiliza la misma ubicación de memoria. Pero cuando el valor cambie, creará un nuevo objeto nuevo y asignará el valor al nuevo objeto

Cadenas StringBuilder

El comportamiento de inmutabilidad de cadenas de C# puede ser muy, muy peligroso cuando se trata de concatenación de cadenas

Para Resolverlo .NET Framework proporciona la clase StringBuilder

Si utiliza el generador de cadenas, no se crearán objetos nuevos cada vez que concatene algo a la variable de cadena en C#

Modifiquemos la concatenación anterior y utilicemos

```
StringBuilder stringBuilder =
new StringBuilder();

stringBuilder.Append("UTN
FRRA");
```

Práctica

Cree un formulario en Visual estudio con dos campos de texto para ingresar valores.

Inserte un botón procesar y un botón cerrar.

Insertemos una etiqueta ->label para mostrar los resultados

El programa deberá ejecutar los algoritmos de cálculo de tiempo utilizados para concatenar con el símbolo más + y los tiempos utilizando la función String Builder.

Cada función llamada recibirá un string como argumento por referencia para poder comunicar la respuesta del método que ejecuta. los demás valores quedan a criterio del desarrollador.